

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN  
**Khoa Toán - Tin học**  
**Bộ môn Ứng dụng Tin học**

# **TOÁN RỜI RẠC 2A**

## **Chương 3. Các thuật toán trên đồ thị**

**GV: Lê Thị Tuyết Nhung**

- 1 Tìm kiếm trên đồ thị
  - Thuật toán tìm kiếm theo chiều sâu (Depth-First Search - DFS)
  - Thuật toán tìm kiếm theo chiều rộng (Breadth-First Search - BFS)
  - Một số ứng dụng của DFS và BFS

# Tìm kiếm trên đồ thị

Một bài toán quan trọng trong lý thuyết đồ thị là bài toán duyệt tất cả các đỉnh có thể đến được từ một đỉnh xuất phát nào đó. Vấn đề này đưa về một bài toán liệt kê mà yêu cầu của nó là không được bỏ sót hay lặp lại bất kỳ đỉnh nào. Vì vậy mà ta phải xây dựng những thuật toán cho phép duyệt một cách hệ thống các đỉnh, những thuật toán như vậy gọi là những thuật toán tìm kiếm trên đồ thị.

Chúng ta xét hai thuật toán cơ bản nhất:

- Thuật toán tìm kiếm theo chiều sâu trên đồ thị.
- Thuật toán tìm kiếm theo chiều rộng trên đồ thị.

## ✳ Nguyên lý

- Bắt đầu tìm kiếm từ một đỉnh  $v_0$  nào đó của đồ thị
- Chọn một đỉnh  $u$  bất kỳ kề với  $v_0$  và lấy nó làm đỉnh duyệt tiếp theo.
- Lặp lại quá trình này với  $u$  cho đến khi không tìm được đỉnh kề tiếp theo nữa thì trở về đỉnh ngay trước đỉnh mà không thể đi tiếp để tìm qua nhánh khác.

# Thuật toán tìm kiếm theo chiều sâu

## Thuật toán (cài đặt đệ quy)

**Bước 1.** Lấy  $s$  là một đỉnh của đồ thị.

**Bước 2.** Đặt  $v = s$ .

**Bước 3.** Duyệt đỉnh  $v$ .

**Bước 4.** Nếu với mọi đỉnh kề của  $v$  đều được duyệt, đặt  $v$  bằng đỉnh đã được duyệt trước đỉnh  $v$ .

Nếu  $v = s$  thì đi đến Bước 6, ngược lại trở lại Bước 3.

**Bước 5.** Chọn  $u$  là đỉnh kề chưa được duyệt của  $v$ , đặt  $v = u$ , trở lại Bước 3.

**Bước 6.** Kết thúc.

# Thuật toán tìm kiếm theo chiều sâu

Thuật toán DFS( $u$ ) có thể được mô tả bằng thủ tục đệ quy như sau:

## Thuật toán DFS ( $u$ )

```
DFS( $u$ ) { //  $u$  là đỉnh bắt đầu duyệt
    <Thăm đỉnh  $u$ >; // Duyệt đỉnh  $u$ 
    chuaxet[ $u$ ] = false; // Xác nhận đỉnh  $u$  đã duyệt
    for  $v \in Ke(u)$  do // Lấy mỗi đỉnh  $v \in Ke(u)$ .
        if (chuaxet[ $v$ ]) then // Nếu đỉnh  $v$  chưa duyệt
            DFS( $v$ ); // Duyệt theo chiều sâu bắt từ đỉnh  $v$ 
        endif
    endfor
}
```

# Thuật toán tìm kiếm theo chiều sâu

## Thuật toán (cài đặt không đệ quy)

**Bước 1.** Lấy  $s$  là một đỉnh của đồ thị.

**Bước 2.** Đặt  $s$  vào STACK.

**Bước 3.** Nếu STACK rỗng đi đến 7.

**Bước 4.** Lấy đỉnh  $p$  từ STACK.

**Bước 5.** Duyệt đỉnh  $p$ .

**Bước 6.** Đặt các đỉnh kề của  $p$  chưa được xét (chưa từng có mặt trong STACK) vào STACK, trở lại 3.

**Bước 7.** Kết thúc.

# Thuật toán tìm kiếm theo chiều sâu

Thuật toán DFS( $u$ ) có thể khử đệ quy bằng cách sử dụng ngăn xếp

## Thuật toán DFS ( $u$ )

DFS( $u$ ) {

**Bước 1: Khởi tạo**

stack =  $\emptyset$ ;    // Khởi tạo stack là  $\emptyset$

push(stack,  $u$ );    // Đưa đỉnh  $u$  vào ngăn xếp

<Thăm đỉnh  $u$ >;    // Duyệt đỉnh  $u$

chuaxet[ $u$ ] = false;    // Xác nhận đỉnh  $u$  đã duyệt



## Bước 2: Lặp

```
while (stack  $\neq \emptyset$ ) do
    s = pop(stack);    //Loại đỉnh ở đầu ngăn xếp
    for  $t \in \text{Ke}(s)$  do    //Lấy mỗi đỉnh  $t \in \text{Ke}(s)$ 
        if (chuaxet[t]) then    //Nếu  $t$  chưa được duyệt
            <Thăm đỉnh  $t$ >;    //Duyệt đỉnh  $t$ 
            chuaxet[t] = false;    //Đỉnh  $t$  đã duyệt
            push(stack, s);    //Đưa  $s$  vào stack
            push(stack, t);    //Đưa  $t$  vào stack
            break;    //Chỉ lấy một đỉnh  $t$ 
        endif
    endfor
endwhile
```

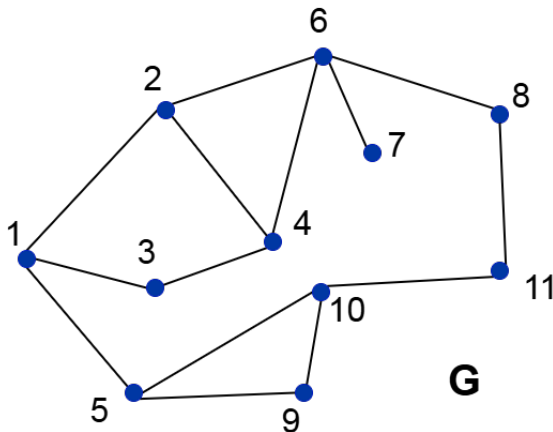
## Bước 3: Trả lại kết quả

```
return(<Tập đỉnh đã duyệt>);
```

```
}
```

# Thuật toán tìm kiếm theo chiều sâu

**Ví dụ:** Kiểm nghiệm thuật toán DFS(1) trên đồ thị  $G$  gồm 11 đỉnh dưới đây?



# Thuật toán tìm kiếm theo chiều sâu

| Đỉnh bắt đầu duyệt | Các đỉnh đã duyệt:<br>$\text{chuaxet}[u] = \text{false}$ | Các đỉnh chưa duyệt<br>$\text{chuaxet}[u] = \text{true}$ |
|--------------------|----------------------------------------------------------|----------------------------------------------------------|
| DFS(1)             | 1                                                        | 2, 3, 4, 5, 6, 7, 8, 9, 10, 11                           |
| DFS(2)             | 1, 2                                                     | 3, 4, 5, 6, 7, 8, 9, 10, 11                              |
| DFS(4)             | 1, 2, 4                                                  | 3, 5, 6, 7, 8, 9, 10, 11                                 |
| DFS(3)             | 1, 2, 4, 3                                               | 5, 6, 7, 8, 9, 10, 11                                    |
| DFS(6)             | 1, 2, 4, 3, 6                                            | 5, 7, 8, 9, 10, 11                                       |
| DFS(7)             | 1, 2, 4, 3, 6, 7                                         | 5, 8, 9, 10, 11                                          |
| DFS(8)             | 1, 2, 4, 3, 6, 7, 8                                      | 5, 9, 10, 11                                             |
| DFS(11)            | 1, 2, 4, 3, 6, 7, 8, 11                                  | 5, 9, 10                                                 |
| DFS(10)            | 1, 2, 4, 3, 6, 7, 8, 11, 10                              | 5, 9                                                     |
| DFS(5)             | 1, 2, 4, 3, 6, 7, 8, 11, 10, 5                           | 9                                                        |
| DFS(9)             | 1, 2, 4, 3, 6, 7, 8, 11, 10, 5, 9                        | $\emptyset$                                              |

**Kết quả duyệt:** 1, 2, 4, 3, 6, 7, 8, 11, 10, 5, 9.

# Thuật toán tìm kiếm theo chiều sâu

| STT | Trạng thái ngăn xếp                   | Danh sách đỉnh được duyệt         |
|-----|---------------------------------------|-----------------------------------|
| 1   | 1                                     | 1                                 |
| 2   | 1, 2                                  | 1, 2                              |
| 3   | 1, 2, 4                               | 1, 2, 4                           |
| 4   | 1, 2, 4, 3                            | 1, 2, 4, 3                        |
| 5   | 1, 2, 4                               | 1, 2, 4, 3                        |
| 6   | 1, 2, 4, 6                            | 1, 2, 4, 3, 6                     |
| 7   | 1, 2, 4, 6, 7                         | 1, 2, 4, 3, 6, 7                  |
| 8   | 1, 2, 4, 6                            | 1, 2, 4, 3, 6, 7                  |
| 9   | 1, 2, 4, 6, 8                         | 1, 2, 4, 3, 6, 7, 8               |
| 10  | 1, 2, 4, 6, 8, 11                     | 1, 2, 4, 3, 6, 7, 8, 11           |
| 11  | 1, 2, 4, 6, 8, 11, 10                 | 1, 2, 4, 3, 6, 7, 8, 11, 10       |
| 12  | 1, 2, 4, 6, 8, 11, 10, 5              | 1, 2, 4, 3, 6, 7, 8, 11, 10, 5    |
| 13  | 1, 2, 4, 6, 8, 11, 10, 5, 9           | 1, 2, 4, 3, 6, 7, 8, 11, 10, 5, 9 |
| 14  | Lần lượt bỏ các đỉnh ra khỏi ngăn xếp |                                   |

**Kết quả duyệt:** 1, 2, 4, 3, 6, 7, 8, 11, 10, 5, 9.

# Thuật toán tìm kiếm theo chiều sâu

**Bài tập:** Cho đồ thị gồm 12 đỉnh được biểu diễn dưới dạng ma trận kề như bên cạnh. Hãy cho biết kết quả thực hiện thuật toán DFS(1). Chỉ rõ trạng thái của ngăn xếp và tập đỉnh được duyệt theo mỗi bước thực hiện của thuật toán.

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

# Thuật toán tìm kiếm theo chiều rộng

## ✳ Nguyên lý

- Bắt đầu tìm kiếm từ một đỉnh  $v$  bất kỳ của đồ thị
- Duyệt tất cả những đỉnh kề của  $v$  lưu vào một tập  $T(u)$  (với đồ thị có hướng thì  $T(u)$  là tập các đỉnh  $u$  với  $u$  là đỉnh sau,  $v$  là đỉnh đầu của cung  $uv$ ).
- Sau đó tiếp tục xét các đỉnh  $u$  thuộc  $T(u)$  và áp dụng lại cách duyệt giống như với  $v$ .

# Thuật toán tìm kiếm theo chiều rộng

## Thuật toán

**Bước 1.** Lấy  $s$  là một đỉnh của đồ thị.

**Bước 2.** Đặt  $s$  vào Queue.

**Bước 3.** Lặp nếu Queue chưa rỗng.

- a. Lấy đỉnh  $p$  từ Queue
- b. Duyệt đỉnh  $p$
- c. Đặt các đỉnh kề của  $p$  chưa được xét (chưa từng có mặt trong Queue) vào Queue
- d. Kết thúc lặp

# Thuật toán tìm kiếm theo chiều rộng

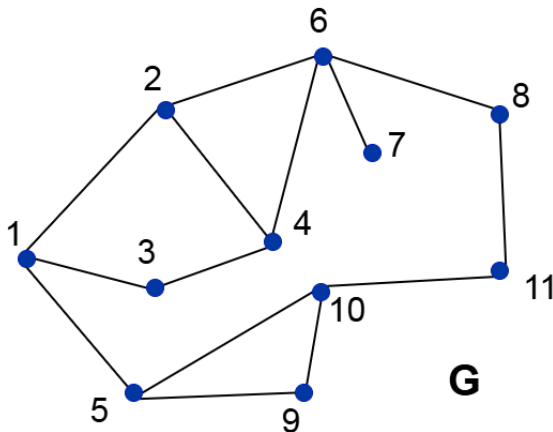
## Thuật toán BFS ( $u$ )

```
BFS( $u$ ){  
    Bước 1: Khởi tạo  
    queue =  $\emptyset$ ; push(queue,  $u$ ); chuaxet[ $u$ ] = false;  
    Bước 2: Lặp  
    while (queue  $\neq \emptyset$ ) do  
         $s$  = pop(queue); <Thăm đỉnh  $s$ >;  
        for  $t \in Ke(s)$  do  
            if (chuaxet[ $t$ ]) then  
                push(queue,  $t$ ); chuaxet[ $t$ ] = false;  
            endif  
        endfor  
    endwhile  
    Bước 3: Trả lại kết quả  
    return(<Tập đỉnh đã duyệt>);  
}
```



# Thuật toán tìm kiếm theo chiều rộng

**Ví dụ:** Kiểm nghiệm thuật toán BFS(1) trên đồ thị  $G$  gồm 11 đỉnh dưới đây?



# Thuật toán tìm kiếm theo chiều rộng

| STT | Trang thái hàng đợi | Danh sách đỉnh được duyệt         |
|-----|---------------------|-----------------------------------|
| 1   | 1                   | $\emptyset$                       |
| 2   | 2, 3, 5             | 1                                 |
| 3   | 3, 5, 4, 6          | 1, 2                              |
| 4   | 5, 4, 6             | 1, 2, 3                           |
| 5   | 4, 6, 9, 10         | 1, 2, 3, 5                        |
| 6   | 6, 9, 10            | 1, 2, 3, 5, 4                     |
| 7   | 9, 10, 7, 8         | 1, 2, 3, 5, 4, 6                  |
| 8   | 10, 7, 8            | 1, 2, 3, 5, 4, 6, 9               |
| 9   | 7, 8, 11            | 1, 2, 3, 5, 4, 6, 9, 10           |
| 10  | 8, 11               | 1, 2, 3, 5, 4, 6, 9, 10, 7        |
| 11  | 11                  | 1, 2, 3, 5, 4, 6, 9, 10, 7, 8     |
| 12  | $\emptyset$         | 1, 2, 3, 5, 4, 6, 9, 10, 7, 8, 11 |

# Thuật toán tìm kiếm theo chiều rộng

**Bài tập:** Cho đồ thị gồm 12 đỉnh được biểu diễn dưới dạng ma trận kề như bên cạnh. Hãy cho biết kết quả thực hiện thuật toán BFS(1). Chỉ rõ trạng thái của hàng đợi và tập đỉnh được duyệt theo mỗi bước thực hiện của thuật toán.

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

# Ứng dụng của thuật toán DFS và BFS

Có rất nhiều ứng dụng khác nhau của thuật toán DFS và BFS trên đồ thị. Những ứng dụng cơ bản của thuật toán DFS và BFS bao gồm:

- o Duyệt tất cả các đỉnh của đồ thị.
- o Duyệt tất cả các thành phần liên thông của đồ thị.
- o Tìm đường đi từ đỉnh  $s$  đến đỉnh  $t$  trên đồ thị.
- o Duyệt các đỉnh khớp/cạnh cầu trên đồ thị vô hướng.
- o Định chiều đồ thị vô hướng.
- o Duyệt các đỉnh rẽ nhánh của cặp đỉnh  $s, t$ .
- o Xác định tính liên thông mạnh/yếu trên đồ thị có hướng.
- o Thuật toán tìm kiếm theo chiều rộng trên đồ thị.
- o Xây dựng cây khung của đồ thị vô hướng liên thông.
- o ...